



**VNiVERSiDAD
D SALAMANCA**

Facultad de Ciencias
Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de una IA para jugar Riichi Mahjong

Development of an AI to Play Riichi Mahjong

Predicción y evaluación estratégica de descartes de Riichi Mahjong mediante Transformers

Prediction and Strategic Evaluation of Riichi Mahjong Discards Using Transformers

ANEXO IV

Documentación Técnica de Programación

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca
Junio 2026

Índice general

1	Introducción	2
2	Estructura del Proyecto	2
2.1	Organización del Repositorio	2
2.2	Jupyter Notebooks	3
3	Módulos de Procesamiento de Datos	3
3.1	Decodificación y Parser	3
3.2	Representación del Estado	4
3.3	Tokenización	4
4	Implementación del Modelo	5
4.1	Definición del Modelo Transformer	5
4.2	Embeddings y Clasificación	6
5	Entrenamiento	6
5.1	Preparación de los Datos	6
5.2	Bucle de Entrenamiento	7
5.3	Checkpoints	7
6	Evaluación	7
6.1	Evaluación Tradicional	7
6.2	Evaluación Estratégica	8
6.3	Medición del Coste de Evaluación	8
7	Dependencias y Entorno de Ejecución	9

1 Introducción

Este documento recoge la documentación técnica de programación del sistema desarrollado durante este Trabajo Fin de Grado. En él se describe la organización del código, los principales módulos implementados y la relación entre estos componentes durante el procesamiento de los datos, el entrenamiento y la evaluación del modelo.

El objetivo de este anexo es facilitar la comprensión de la implementación del proyecto y permitir identificar las partes del código responsables de cada una de las funcionalidades descritas en los anexos anteriores. Además, se describe el entorno y las principales dependencias necesarias para la ejecución del proyecto.

2 Estructura del Proyecto

2.1 Organización del Repositorio

El repositorio se organiza separando el código fuente, los datos, los experimentos y los resultados generados durante el proyecto. Esta separación permite mantener los diferentes componentes del sistema de forma independiente y facilita la realización de nuevas iteraciones.

La estructura principal del repositorio es la siguiente:

```
.
|-- checkpoints/
|-- data/
|-- notebooks/
|-- reports/
|-- src/
|   |-- analysis/
|   |-- configs/
|   |-- data/
|   |-- features/
|   |-- models/
|   |-- training/
|   `-- utils/
|-- environment.yml
|-- requirements.txt
`-- README.md
```

El directorio `src/` contiene la implementación principal del sistema y se divide en módulos según su responsabilidad. Por otro lado, `notebooks/` contiene los experimentos realizados durante las diferentes etapas del proyecto. Los modelos entrenados se almacenan en `checkpoints/`, mientras que los resultados, tablas y métricas generadas durante la evaluación se encuentran en `reports/`.

Finalmente, `data/` contiene los datos originales y las diferentes versiones generadas durante su procesamiento.

2.2 Jupyter Notebooks

Los Jupyter Notebooks fueron utilizados como principal entorno de experimentación durante el desarrollo. Cada notebook corresponde a una etapa o experimento concreto y contiene tanto el código utilizado como los resultados y observaciones obtenidas durante su ejecución.

Los principales notebooks del proyecto son:

Cuadro 1: Jupyter Notebooks utilizados durante el desarrollo

Notebook	Propósito
01_eda.ipynb	Análisis exploratorio inicial del dataset.
02_representation_design.ipynb	Diseño y validación de la representación de los estados de juego.
03_model_baselines.ipynb	Desarrollo y entrenamiento de los primeros modelos base.
04_large_scale_training.ipynb	Entrenamiento del modelo utilizando una mayor cantidad de datos.
05_policy_analysis.ipynb	Análisis de las predicciones y comportamiento del modelo.
07_strategic_dataset_training.ipynb	Entrenamiento de los modelos utilizados para la evaluación estratégica.
08_validation_analysis_100k.ipynb	Evaluación estratégica del modelo entrenado con 100k estados.
09_validation_analysis_500k.ipynb	Evaluación estratégica del modelo entrenado con 500k estados.
10_model_comparison.ipynb	Comparación final entre los modelos de 100k y 500k estados.

Esta organización permite mantener separados los diferentes experimentos y conservar los resultados obtenidos durante cada iteración del proyecto.

3 Módulos de Procesamiento de Datos

3.1 Decodificación y Parser

Los datos originales utilizados por el proyecto contienen diferentes estructuras numéricas para representar las piezas, acciones y combinaciones realizadas durante una partida. Para facilitar su interpretación y comprobar el correcto procesamiento de los datos, se implementaron diferentes módulos de decodificación.

El módulo `tile_decoder.py` contiene las funciones necesarias para convertir los identificadores de piezas utilizados por el dataset a la representación de 34 tipos de piezas utilizada durante diferentes partes del proyecto. Además, permite obtener representaciones legibles de las piezas para facilitar la depuración y análisis de los estados.

Por otro lado, `action_decoder.py` y `meld_decoder.py` permiten interpretar las acciones legales y las combinaciones presentes en los estados. Estos módulos fueron utilizados principalmente durante las primeras etapas del desarrollo para comprobar que la información extraída del dataset correspondía correctamente con las acciones de Riichi Mahjong.

Finalmente, `state_decoder.py` permite visualizar de forma legible una instantánea decodificada, incluyendo información de la ronda, mano, jugadores, descartes, combinaciones y acciones disponibles. Esto permitió inspeccionar manualmente los ejemplos durante el desarrollo y comprobar el funcionamiento del procesamiento de los datos.

3.2 Representación del Estado

La transformación de los datos originales a la representación utilizada por el sistema se implementa principalmente en el módulo `state_representation.py`.

La función principal, `extract_state_features()`, recibe un estado procedente del dataset y genera una estructura canónica con la información necesaria para las siguientes etapas del procesamiento.

Durante este proceso, los identificadores de las piezas son convertidos a una representación basada en los 34 tipos diferentes de piezas de Mahjong. Además, la información de cada jugador y de las acciones disponibles se extrae mediante funciones auxiliares independientes.

El estado resultante contiene la información global de la partida, los indicadores de Dora, la mano del jugador, el estado de los cuatro jugadores, las acciones válidas y el índice correspondiente a la decisión realizada por el jugador real.

Esta representación funciona como una capa intermedia entre los datos originales y los componentes posteriores del sistema, evitando que el modelo dependa directamente de la estructura utilizada por el dataset.

3.3 Tokenización

La tokenización se implementa mediante la clase `MahjongTokenizer`, encargada de transformar un estado estructurado en una secuencia de tokens semánticos.

En lugar de representar todos los elementos del estado mediante un único tipo de token, se definieron diferentes estructuras dependiendo de la información representada. Para ello, el módulo `token_definitions.py` define los siguientes tipos principales:

- `GameStateToken`: información global de la partida.
- `PlayerStateToken`: información correspondiente a cada jugador.
- `HandToken`: piezas presentes en la mano.
- `DiscardToken`: descartes realizados durante la partida.
- `MeldToken`: combinaciones abiertas realizadas por los jugadores.

- `ActionToken`: acciones legales disponibles en el estado.

La función `tokenize()` genera cada uno de estos grupos y los combina para formar la secuencia completa utilizada posteriormente por el modelo.

Además de la información propia de las piezas, algunos tokens intermedios almacenan información adicional necesaria para conservar su significado dentro del estado. Por ejemplo, los descartes contienen el jugador que realizó la acción, si fue un *tsumogiri* y su orden dentro de la secuencia de descartes. De forma similar, los tokens de combinaciones almacenan su tipo, las piezas involucradas y los jugadores de procedencia. Sin embargo, la tensorización final solo transmite al modelo el tipo de token, la pieza, su copia, el jugador, el tipo de acción y la posición absoluta.

De esta forma, la secuencia generada mantiene separadas las diferentes categorías semánticas presentes en un estado de Riichi Mahjong.

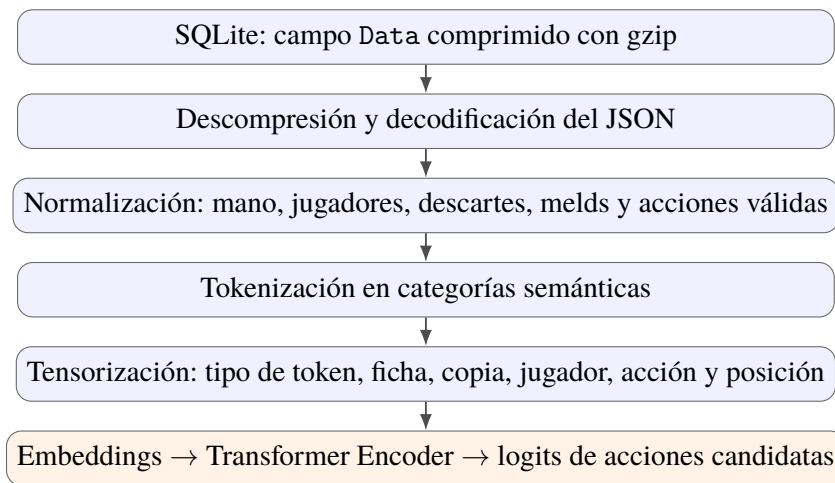


Figura 1: Flujo implementado para procesar una instantánea del dataset hasta obtener la puntuación de las acciones candidatas.

4 Implementación del Modelo

4.1 Definición del Modelo Transformer

La implementación principal del modelo se encuentra en el módulo `transformer_model.py`, mediante la clase `MahjongTransformer`. Esta clase utiliza los componentes de Transformers disponibles en PyTorch para construir el modelo utilizado durante los experimentos.

La configuración utilizada por defecto está formada por un tamaño de embedding de 128 dimensiones, 8 cabezas de atención, 4 capas de Transformer Encoder, una dimensión de 512 para las capas *feed-forward* y un *dropout* de 0.1.

El modelo está compuesto principalmente por tres etapas. Primeramente, la capa `MahjongEmbedding` transforma los datos de entrada en representaciones vectoriales. A continuación, estas representaciones son procesadas por el `TransformerEncoder`. Finalmente, se seleccionan los estados ocultos correspondientes a los tokens de las acciones candidatas y se procesan mediante una capa lineal denominada `policy_head`.

La capa `policy_head` transforma la representación de cada acción en un único valor o *logit*. Por lo tanto, la salida del modelo está formada por una puntuación para cada una de las acciones candidatas del estado.

4.2 Embeddings y Clasificación

La generación de los embeddings se encuentra implementada en `embeddings.py` mediante la clase `MahjongEmbedding`. Esta clase mantiene diferentes tablas de embeddings dependiendo de la información representada por cada token.

Concretamente, se utilizan embeddings para el tipo de token, la pieza, la copia de la pieza, el jugador, el tipo de acción y la posición dentro de la secuencia. Todos estos embeddings utilizan la misma dimensión y son sumados para obtener la representación final de cada token.

La configuración completa contiene 867.073 parámetros entrenables. La implementación evaluada no incorpora embeddings específicos para otros metadatos presentes en los objetos intermedios, como puntuaciones, estado de Riichi, Honba, *tsumogiri* o detalles completos de las combinaciones.

Esta representación es utilizada posteriormente como entrada del Transformer Encoder. Durante el procesamiento también se utiliza una máscara de atención para evitar que los elementos de padding introducidos durante la creación de los batches afecten al resultado del modelo.

Una vez obtenida la salida del Transformer, se utilizan los índices de los tokens correspondientes a las acciones candidatas para extraer sus representaciones. La capa `policy_head` genera un logit para cada una de ellas, permitiendo posteriormente seleccionar la acción con mayor puntuación.

5 Entrenamiento

5.1 Preparación de los Datos

La preparación de los datos para el entrenamiento se realiza principalmente mediante las clases `MahjongDataset` y `MahjongCollator`.

La clase `MahjongDataset` permite acceder directamente a los estados almacenados en la base de datos SQLite. Para cada muestra, se recupera el campo `Data` y se descomprime, decodifica y normaliza la instantánea utilizada durante el entrenamiento. A continuación, esta pasa por el tokenizer y por el proceso de tensorización antes de ser devuelta al modelo.

Además, la clase permite seleccionar una tabla concreta del dataset, así como limitar el número de muestras y definir un desplazamiento dentro de la base de datos. Esto facilita la creación de diferentes conjuntos de entrenamiento, validación y test.

Una vez procesadas las muestras individuales, `MahjongCollator` se encarga de agruparlas en batches. Debido a que los estados pueden generar secuencias de diferente longitud, el collator añade padding hasta igualar todas las secuencias dentro de un mismo batch y genera la correspondiente máscara de atención.

Los índices de las acciones candidatas se mantienen con longitud variable, ya que el número de acciones disponibles puede ser diferente para cada estado.

5.2 Bucle de Entrenamiento

El proceso principal de entrenamiento se encuentra implementado en la función `train_model()`, definida en el módulo `train_loop.py`.

Para cada época, el modelo se establece en modo de entrenamiento y se recorren todos los batches disponibles. Primeramente, los datos son transferidos al dispositivo de ejecución. A continuación, se realiza el forward pass del modelo para obtener los logits correspondientes a las acciones candidatas de cada muestra.

Debido a que cada estado puede contener un número diferente de acciones candidatas, la función de pérdida se calcula individualmente para cada muestra del batch. Posteriormente, las pérdidas se promedian para obtener la pérdida total del batch.

A partir de esta pérdida se realiza el proceso de backpropagation y la actualización de los parámetros mediante el optimizador. Al mismo tiempo, se calcula la exactitud de entrenamiento comparando la acción con mayor puntuación con la acción real almacenada en el dataset.

El bucle permite evaluar un conjunto de validación y almacenar la pérdida, la exactitud y las exactitudes Top-3 y Top-5. Esta funcionalidad no se utilizó de forma homogénea en todos los experimentos. La comparación definitiva se realizó posteriormente sobre los últimos 52.563 estados, separados por posición en la base de datos de los 500.000 empleados para el segundo entrenamiento.

5.3 Checkpoints

Para conservar el progreso del entrenamiento se implementó un sistema de checkpoints mediante el módulo `checkpointing.py`.

Al finalizar cada época, el sistema puede guardar un fichero que contiene el estado actual del modelo, el estado del optimizador y el número de época correspondiente. Esto permite conservar las diferentes versiones del modelo generadas durante el entrenamiento y facilita continuar el proceso desde un punto anterior.

Los checkpoints se almacenan utilizando el formato de serialización proporcionado por PyTorch y se identifican mediante el número de época.

6 Evaluación

6.1 Evaluación Tradicional

La evaluación tradicional del modelo se encuentra implementada principalmente en los módulos `evaluate.py` y `metrics.py`. Estas funciones permiten evaluar el modelo sobre un conjunto de datos independiente sin realizar modificaciones sobre sus parámetros.

La función `evaluate_model()` recorre los diferentes batches del conjunto de evaluación y obtiene las predicciones del modelo. Para cada muestra se calcula la función de pérdida y se compara la acción con mayor puntuación con la acción realizada por el jugador real.

Además de la exactitud, se calculan las métricas Top-3 Accuracy y Top-5 Accuracy mediante la función `compute_topk_accuracy()`. Estas métricas permiten comprobar si la acción realizada por el jugador se encuentra entre las acciones con mayor puntuación asignadas por el modelo.

Finalmente, el evaluador devuelve la pérdida media, la exactitud y los valores de Top-3 y Top-5 Accuracy obtenidos sobre el conjunto evaluado.

6.2 Evaluación Estratégica

La evaluación estratégica se implementa mediante diferentes módulos encargados de calcular el Shanten, el Ukeire y clasificar posteriormente las diferencias entre la decisión del modelo y la decisión realizada por el jugador real.

El módulo `shanten.py` utiliza la biblioteca `mahjong` para calcular el Shanten de una mano. Además, permite evaluar los posibles descartes de una mano y obtener el Shanten resultante después de cada uno de ellos.

Por otro lado, `ukeire.py` implementa el cálculo del Ukeire efectivo. Para ello, primeramente se identifican las piezas visibles en el estado, incluyendo la propia mano, los indicadores de Dora, los descartes y las combinaciones de los jugadores. A partir de esta información se estima el número de copias restantes de cada pieza.

Para cada tipo de pieza todavía disponible se comprueba si su incorporación a la mano produce una mejora del Shanten. En caso afirmativo, las copias restantes de dicha pieza son añadidas al Ukeire efectivo. De esta forma, la métrica tiene en cuenta no solamente qué piezas permiten mejorar la mano, sino también cuántas de ellas continúan disponibles según la información visible.

Finalmente, el módulo `error_analysis.py` compara el Shanten y Ukeire obtenidos por la predicción del modelo y por la decisión del jugador. A partir de esta comparación, cada predicción es asignada a una categoría estratégica que permite analizar con mayor detalle el comportamiento del modelo.

Cuadro 2: Categorías utilizadas durante la evaluación estratégica

Categoría	Condición
<code>exact_match</code>	El modelo selecciona la misma acción que el jugador.
<code>better_shanten</code>	La decisión del modelo produce un Shanten menor.
<code>shanten_loss</code>	La decisión del modelo produce un Shanten mayor.
<code>equivalent</code>	Ambas decisiones producen el mismo Shanten y Ukeire.
<code>near_equivalent</code>	Ambas decisiones mantienen el mismo Shanten y presentan una diferencia de Ukeire igual o inferior a 2 piezas.
<code>better_ukeire</code>	Se mantiene el mismo Shanten y el modelo obtiene una mejora de Ukeire superior a 2 piezas.
<code>ukeire_loss</code>	Se mantiene el mismo Shanten y el modelo obtiene una pérdida de Ukeire superior a 2 piezas.

6.3 Medición del Coste de Evaluación

La ejecución final se cronometró sobre 52.563 estados, utilizando lotes de 64 muestras, una GPU NVIDIA GeForce RTX 5060 Laptop y los dos checkpoints evaluados de forma sucesiva. La inferencia predictiva produjo 105.126 predicciones en 167 segundos, equivalentes a 629 predicciones por segundo. El cálculo posterior de Shanten, Ukeire y categorías estratégicas necesitó 481 segundos, con un rendimiento de 109 estados por segundo.

En conjunto, ambas etapas necesitaron aproximadamente 10 minutos y 48 segundos. El 74,2% del tiempo correspondió a la evaluación heurística, mientras que la inferencia de los

dos Transformers representó el 25,8%. Estas cifras proceden de una única ejecución y sirven como referencia del orden de magnitud, no como un benchmark independiente del hardware. Cada checkpoint ocupa aproximadamente 10,5 MB y el modelo contiene 867.073 parámetros entrenables.

7 Dependencias y Entorno de Ejecución

El entorno utilizado durante el desarrollo del proyecto se encuentra definido mediante el fichero `environment.yml`. Este fichero permite recrear el entorno Conda utilizado para ejecutar los notebooks, entrenar los modelos y realizar los diferentes procesos de evaluación.

El entorno recibe el nombre de `riichi-ai` y utiliza Python 3.11. Entre las principales dependencias utilizadas durante el proyecto se encuentran PyTorch para la implementación y entrenamiento del modelo, CUDA para la aceleración mediante GPU, NumPy y Pandas para el procesamiento de los datos, y Matplotlib y Seaborn para la generación de gráficos.

Además, el entorno incluye JupyterLab y las diferentes dependencias necesarias para la ejecución de Jupyter Notebooks, utilizados como principal entorno de experimentación durante el desarrollo.

Las principales dependencias del proyecto se resumen en la Tabla 3.

Cuadro 3: Principales dependencias utilizadas durante el proyecto

Dependencia	Versión	Uso principal
Python	3.11.15	Lenguaje principal utilizado durante el desarrollo.
PyTorch	2.6.0	Implementación y entrenamiento del modelo Transformer.
CUDA	12.9	Aceleración de las operaciones realizadas mediante GPU.
NumPy	2.4.6	Procesamiento y manipulación de datos numéricos.
Pandas	3.0.3	Análisis y procesamiento de los resultados experimentales.
Matplotlib	3.10.9	Generación de gráficos y visualización de resultados.
Seaborn	0.13.2	Visualización estadística de los resultados.
Scikit-learn	1.8.0	Herramientas auxiliares para análisis y evaluación.
JupyterLab	4.5.7	Entorno utilizado para el desarrollo de los experimentos.

Para recrear el entorno se puede utilizar directamente el fichero proporcionado junto con el código del proyecto mediante Conda:

```
conda env create -f environment.yml
conda activate riichi-ai
```

De esta forma, las versiones de las principales dependencias utilizadas durante el desarrollo quedan definidas junto con el propio proyecto, facilitando la reproducción de los experimentos realizados.